

ARTIFICIAL INTELLIGENCE
UNIT-1 ASSESSMENT TOOL
ANALYTICAL PROBLEM SOLVING

Course code and name: CSA1717-Artificial Intelligence

Student Name: S. Harish

Register Number: 192511119

Faculty: Dr. Anoop. M

Question 1: Water Jug Problem

Problem Statement

The Water Jug Problem is a classic Artificial Intelligence search problem. Two jugs with capacities of **4 gallons** and **3 gallons** are given. Neither jug has any measuring marks. There is an unlimited supply of water available through a pump.

The objective is to obtain **exactly 2 gallons of water in the 4-gallon jug** using the following operations:

- Fill any jug completely from the pump.
- Empty any jug onto the ground.
- Pour water from one jug into another until either the first jug is empty or the second jug becomes full.

No other measuring device is allowed.

Problem Formulation

Artificial Intelligence problems are represented using four components:

1. Initial State

Initially, both jugs are empty.

4-Gallon Jug 3-Gallon Jug

0 Gallons 0 Gallons

State Representation:

(0,0)

2. State Space

A state is represented as:

(Amount in 4-gallon jug, Amount in 3-gallon jug)

Example:

(4,3) → Both jugs are full

(2,0) → Goal State

(1,3) → 1 gallon in 4-gallon jug and 3 gallons in 3-gallon jug

3. Operators (Production Rules)

The possible actions are:

1. Fill the 4-gallon jug.
2. Fill the 3-gallon jug.
3. Empty the 4-gallon jug.
4. Empty the 3-gallon jug.
5. Pour water from the 4-gallon jug to the 3-gallon jug.
6. Pour water from the 3-gallon jug to the 4-gallon jug.

4. Goal Test

The goal is achieved when the 4-gallon jug contains exactly **2 gallons**.

Goal State:

(2,0)

Step-by-Step Solution

Step	Operation	4-Gallon Jug	3-Gallon Jug
Initial	Both Empty	0	0
1	Fill 3-gallon jug	0	3
2	Pour 3 into 4	3	0

3	Fill 3-gallon jug again	3	3
4	Pour from 3 to 4 until 4 is full	4	2
5	Empty 4-gallon jug	0	2
6	Pour remaining 2 gallons into 4-gallon jug	2	0

State Transition Table

Current State	Action	Next State
(0,0)	Fill 3-gallon jug	(0,3)
(0,3)	Pour to 4-gallon jug	(3,0)
(3,0)	Fill 3-gallon jug	(3,3)
(3,3)	Pour into 4-gallon jug	(4,2)
(4,2)	Empty 4-gallon jug	(0,2)
(0,2)	Pour into 4-gallon jug	(2,0)

Algorithm

Step 1: Start with both jugs empty

Step 2: Fill the 3-gallon jug.

Step 3: Pour all water into the 4-gallon jug.

Step 4: Fill the 3-gallon jug again.

Step 5: Pour water into the 4-gallon jug until it becomes full.

Step 6: Empty the 4-gallon jug.

Step 7: Transfer the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.

Step 8: Stop.

Pseudocode

Algorithm Water Jug():

Start

Fill (3)

Pour (3 → 4)

Fill (3)

Pour (3 $\rightarrow$ 4 until 4 is full)

Empty (4)

Pour (3 $\rightarrow$ 4)

If 4-gallon jug contains 2 gallons

 Goal Reached

Else

 Continue searching

End

Advantages of Using AI Search

- Systematically explores all possible states.
- Guarantees finding the solution when one exists.
- Easily represented using graphs and state spaces.
- Demonstrates concepts of state-space search and problem-solving agents.

Applications of the Water Jug Problem

- Artificial Intelligence problem solving.
- Robotics planning.
- Automated reasoning.
- State-space search algorithms.
- Game AI and puzzle solving.

Conclusion

The Water Jug Problem demonstrates how Artificial Intelligence models a real-world problem using **states, operators, goal tests, and search strategies**. By applying valid operations in sequence, the agent reaches the goal state **(2,0)**, where the 4-gallon jug contains exactly **2 gallons**. This problem illustrates the fundamentals of AI search techniques and problem-solving agents.

Question 2: Mars Rover – Intelligent Agent for Planetary Exploration

Problem Statement

A **Mars Rover** is an autonomous robotic vehicle designed by space agencies such as NASA to explore the surface of Mars. Since communication between Earth and Mars takes several minutes, the rover must make many decisions on its own. It collects scientific data, analyzes rocks and soil, captures images, avoids obstacles, and transmits valuable information back to Earth.

The Mars Rover is an excellent example of an **Intelligent Agent** in Artificial Intelligence because it perceives its environment using sensors, reasons about its surroundings, and performs actions to achieve mission goals.

Introduction

Artificial Intelligence enables machines to perform tasks that normally require human intelligence. The Mars Rover uses AI to:

- Navigate across rough terrain.
- Detect obstacles.
- Collect and analyse samples.
- Make autonomous decisions.
- Communicate scientific observations to Earth.

Since Mars is an unknown and unpredictable environment, the rover continuously observes its surroundings and adapts its actions based on new information.

i. What are the Percepts for this Agent?

Percepts

Percepts are the information received from the environment through the rover's sensors. These inputs help the rover understand its surroundings and make intelligent decisions.

Major Percepts

Sensor	Percept Received
Navigation Camera	Images of terrain
Hazard Camera	Obstacles and rocks
Panoramic Camera	Wide-angle surface images
Temperature Sensor	Surface temperature
Pressure Sensor	Atmospheric pressure

Wind Sensor	Wind direction and speed
Radiation Sensor	Radiation levels
Spectrometer	Chemical composition of rocks
Drill Sensor	Rock hardness
Battery Sensor	Battery level
Wheel Encoder	Distance travelled
Gyroscope	Orientation and balance
Solar Panel Monitor	Charging status

Examples of Percepts

- Images of Martian rocks
- Soil colour
- Surface texture
- Dust storms
- Hills and valleys
- Battery percentage
- Wheel slippage
- Atmospheric conditions
- Scientific measurements
- GPS/Localization information

ii. Characterize the Operating Environment

The operating environment of the Mars Rover can be analysed using the standard AI environment properties.

Environment Property	Type	Explanation
Observability	Partially Observable	The rover cannot observe the entire Martian environment at once.
Number of Agents	Single Agent	Only one rover performs the mission independently.

Determinism	Stochastic	Unexpected events like dust storms and loose soil may occur.
Episodes	Sequential	Each decision affects future actions.
Dynamics	Dynamic	The environment changes continuously.
State Space	Continuous	Movement and sensor values are continuous.
Knowledge	Unknown	The rover explores previously unknown regions.

iii. What Actions Can the Agent Take?

The rover performs several actions based on its precepts.

Navigation Actions

- Move forward
- Move backward
- Turn left
- Turn right
- Stop

Scientific Actions

- Capture images
- Drill rocks
- Collect soil samples
- Analyse rock composition
- Measure atmospheric conditions
- Record environmental data

Communication Actions

- Send images to Earth
- Send scientific reports
- Receive commands from mission control

Maintenance Actions

- Recharge battery using solar panels
- Monitor battery level

- Check wheel condition
- Perform system diagnostics

Safety Actions

- Detect obstacles
- Avoid dangerous terrain
- Prevent collisions
- Select alternative paths

iv. How Can One Evaluate the Performance of the Agent?

The performance of the Mars Rover is evaluated based on how successfully it completes its mission.

Performance Measures

Performance Metric	Description
Mission Completion	Successfully reaches target locations
Scientific Data	Amount and quality of collected data
Battery Efficiency	Uses minimum energy
Safety	Avoids collisions and hazards
Navigation Accuracy	Follows planned routes
Communication	Successfully transmits information
Sample Collection	Collects maximum useful samples
Time Efficiency	Completes tasks within planned time

Good Performance Means

- Maximum scientific discoveries.
- Minimum energy consumption.
- Safe navigation.
- Continuous communication.
- Successful completion of exploration tasks.

PEAS Description of Mars Rover

PEAS stands for **Performance Measure, Environment, Actuators, and Sensors**.

Component	Description
Performance Measure	Collect maximum scientific data, avoid obstacles, conserve battery, complete mission successfully
Environment	Martian surface, rocks, craters, hills, atmosphere, dust storms
Actuators	Wheels, robotic arm, drill, camera controls, communication antenna
Sensors	Camera, spectrometer, temperature sensor, pressure sensor, gyroscope, battery monitor, radiation sensor

v. Suitable Agent Architecture

The most suitable architecture for the Mars Rover is a **Utility-Based Learning Agent**.

Reason

A simple reflex agent is not sufficient because the Martian environment is unpredictable. The rover must reason about future consequences, adapt to changing conditions, and learn from previous experiences.

A Utility-Based Learning Agent:

- Maintains an internal model of the environment.
- Chooses actions that maximize scientific value.
- Learns from previous experiences.
- Optimizes energy usage.
- Plans safer and more efficient route

Working of the Mars Rover Agent

1. Sensors observe the surrounding environment.
2. The rover receives precepts such as images, temperature, and rock composition.
3. The internal model updates the current state of the environment.
4. The utility function evaluates possible actions.
5. The learning component improves future decisions using previous experiences.
6. The decision-making module selects the best action.
7. Actuators execute the selected action.
8. The cycle repeats until the mission is completed.

Advantages of Using AI in Mars Rover

- Autonomous navigation.
- Intelligent obstacle avoidance.
- Reduced human intervention.
- Efficient energy management.
- Better scientific exploration.
- Real-time decision making.
- Ability to operate in hazardous environments.

Applications

- Planetary exploration.
- Space missions.
- Autonomous robotics.
- Geological surveys.
- Environmental monitoring.
- Disaster response robots.
- Underwater exploration.

Conclusion

The Mars Rover is an excellent example of an **Intelligent Agent** in Artificial Intelligence. It uses sensors to perceive the Martian environment, processes the collected information using AI techniques, and performs intelligent actions to achieve mission objectives. A **Utility-Based Learning Agent architecture** is the most suitable because it allows the rover to learn from experience, adapt to uncertain conditions, maximize scientific output, and safely explore the Martian surface with minimal human intervention.

Question 3: 8-Queens Problem – Problem Formulation and Search Strategies

Problem Statement

The **8-Queens Problem** is one of the most popular problems in Artificial Intelligence and Computer Science. The objective is to place **8 queens on an 8×8 chessboard** such that **no two queens attack each other**.

A queen in chess can attack any piece located:

- Horizontally (same row)
- Vertically (same column)
- Diagonally (both left and right diagonals)

Therefore, a valid solution must satisfy the following conditions:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens lie on the same diagonal.

The problem is commonly solved using **state-space search**, **backtracking**, **depth-first search**, and **heuristic search algorithms**.

Introduction

The 8-Queens problem is an example of a **Constraint Satisfaction Problem (CSP)**. A Constraint Satisfaction Problem consists of:

- Variables
- Domains
- Constraints

Here,

- Variables $\rightarrow$ Columns of the chessboard
- Domain $\rightarrow$ Rows (1–8)
- Constraints $\rightarrow$ Queens must not attack each other

The AI agent searches through different board configurations until it finds a valid solution.

Problem Formulation

Artificial Intelligence problems are represented using the following components.

1. Initial State

Initially, the chessboard is empty.

```
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
```

2. State Representation

Each state represents a partial placement of queens.

Example

```
Q □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ Q □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ Q □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
□ □ □ □ □ □ □ □
```

Each new state is generated by placing one additional queen.

3. Operators

The possible operators are:

- Place a queen in an empty column.
- Move a queen to another safe position.
- Remove a queen (during backtracking).
- Generate a new valid state.

4. Goal Test

The goal is achieved when:

- Eight queens are placed.
- No queen attacks another queen.

Goal State:

8 Queens

0 Conflicts

5. Path Cost

Each queen placement is considered one step.

Total cost

8 queen placements

State Space

Since every column has 8 possible rows,

Total possible configuration

8^8

$= 16,777,216$

However,

Only **92 valid solutions** exist.

This large search space demonstrates why intelligent search strategies are necessary.

Constraints

A queen attacks another queen if:

Same Row

Q □ □ □ □ □ □ □

Q □ □ □ □ □ □ □

Invalid

Same Column

Q

Q

Invalid

Same Diagonal

Q

Q

Invalid

Search Strategies

Several AI search algorithms can solve the 8-Queens problem.

1. Breadth First Search (BFS)

Working

- Explores all nodes level by level.
- Expands every possible state.

Advantages

- Complete.
- Finds the shortest solution.

Disadvantages

- Very high memory usage.
- Slow for large search spaces.

2. Depth First Search (DFS)

Working

- Explores one branch completely.
- Backtracks when conflicts occur.

Advantages

- Requires less memory.
- Faster than BFS.

Disadvantages

- May reach dead ends.

3. Backtracking Search

Backtracking is the most common algorithm for solving the 8-Queens problem.

Steps

1. Place first queen.
2. Check whether position is safe.
3. Place next queen.
4. If conflict occurs, remove previous queen.
5. Try another position.
6. Repeat until all queens are placed.

4. Hill Climbing Search

Hill Climbing is a heuristic search technique.

Working:

- Start with a random arrangement.
- Count attacking queen pairs.
- Move queens to reduce conflicts.
- Stop when no better move exists.

Advantages

- Fast.
- Uses less memory.

Disadvantages

- May get stuck in local maxima.

5. Genetic Algorithm

A Genetic Algorithm treats every chessboard arrangement as a chromosome.

Operations:

- Selection
- Crossover
- Mutation

Advantages

- Excellent for large N-Queens problems.
- Produces solutions quickly.

Algorithm (Backtracking)

Algorithm EightQueens()

Start

Place Queen in Column 1

For each row

 If Safe

 Place Queen

 Recur for next column

 Else

 Try next row

If all columns completed

 Solution Found

Else

 Backtrack

Stop

Pseudocode

procedure Solve(column)

 if column > 8

 print solution

 else

 for each row

 if Safe(row,column)

 place queen

 Solve(column+1)

 remove queen

Example Solution

One valid solution is

Column	1	2	3	4	5	6	7	8
Row	1	5	8	6	3	7	2	4

Chessboard Representation

Q □ □ □ □ □ □ □

□ □ □ □ □ □ Q □

□ □ □ □ Q □ □ □

□ □ □ □ □ □ □ Q

□ Q □ □ □ □ □ □

□ □ □ Q □ □ □ □

□ □ □ □ □ Q □ □

□ □ Q □ □ □ □ □

No queen attacks another queen.

Applications

The 8-Queens problem is used in:

- Artificial Intelligence
- Constraint Satisfaction Problems
- Robotics
- Scheduling Problems
- Resource Allocation
- Optimization Problems
- Game Development

Advantages of AI Search in 8-Queens

- Reduces unnecessary search.
- Finds valid solutions efficiently.
- Demonstrates state-space exploration.
- Applies heuristic optimization.
- Solves complex combinatorial problems.

Conclusion

The **8-Queens Problem** is a classic **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. The objective is to place eight queens on an 8×8 chessboard so that no two queens attack each other. By formulating the problem using **states, operators, goal tests, and path costs**, AI search algorithms such as **Depth-First Search, Breadth-First Search, Backtracking, Hill Climbing, and Genetic Algorithms** can efficiently explore the search space and find valid solutions. Among these, **Backtracking** is the most commonly used and efficient approach because it eliminates invalid configurations early, reducing the search effort significantly.

Question 4: OLA Cab Booking – Problem Solving Agent

Problem Statement

A customer wants to travel from one location to another using the **OLA Cab Booking Mobile Application**. The customer enters the pickup and destination locations and selects a preferred cab type such as **Mini, Micro, Sedan, Prime, Shared**, etc. Based on the customer's preferences, the application searches for available drivers, calculates the fare, estimates the arrival time (ETA), and books the cab.

This problem can be modeled as an **Artificial Intelligence Problem-Solving Agent**, where the system searches through available alternatives and selects the most suitable cab to achieve the user's goal.

Introduction

Artificial Intelligence is widely used in transportation services to automate decision-making. OLA Cab uses AI techniques such as route optimization, demand prediction, fare estimation, driver allocation, and shortest-path algorithms to provide efficient ride-booking services.

The main objective of the OLA application is to provide a **safe, economical, and comfortable ride** while minimizing waiting time and travel cost.

Type of Problem-Solving Agent

The OLA Cab Booking System is best represented by a **Goal-Based Problem Solving Agent**.

A Goal-Based Agent:

- Has a clearly defined goal.
- Searches for possible solutions.
- Evaluates available alternatives.
- Selects the best solution to achieve the goal.

Goal

Book the most suitable cab and transport the customer safely to the destination.

Problem Formulation

AI problems are formulated using the following components.

1. Initial State

The customer opens the OLA application.

The app requests:

- Pickup location
- Destination
- Preferred cab type

2. State Space

Each state represents a stage of the booking process.

Example:

App Opened

↓

Pickup Entered

↓

Destination Entered

↓

Cab Selected

↓

Driver Assigned

↓

Ride Started

↓

Destination Reached

3. Actions (Operators)

The OLA system can perform the following actions:

- Accept pickup location
- Accept destination
- Display available cab types

- Search nearby drivers
- Calculate fare
- Estimate arrival time (ETA)
- Assign nearest driver
- Confirm booking
- Start ride
- Complete ride
- Generate payment receipt

4. Goal Test

The goal is achieved when:

- Driver is assigned successfully.
- Customer reaches the destination.
- Payment is completed.

Goal State:

Ride Successfully Completed

5. Path Cost

The system aims to minimize:

- Travel distance
- Waiting time
- Fare
- Fuel consumption
- Travel time

State Space Diagram

Customer Opens App

|



Enter Pickup Location

|



Enter Destination

|



Choose Cab Type

|



Search Nearby Drivers

|



Driver Available?

/ \

Yes No

|

|



Show Fare Retry Search

|



Customer Confirms?

/ \

Yes No

|

|



Book Cab Cancel Ride

|



Ride Starts

|



Reach Destination

|



Payment

|



Ride Completed

PEAS Description

PEAS stands for **Performance Measure, Environment, Actuators, and Sensors**.

Component	Description
Performance Measure	Minimum waiting time, lowest fare, shortest route, customer satisfaction
Environment	Roads, traffic, drivers, customers, GPS network, weather conditions
Actuators	Mobile application interface, notifications, payment system, GPS navigation
Sensors	GPS location, internet connectivity, driver location, customer location, traffic information

Working of the OLA Cab Booking Agent

The working process is explained below:

Step 1

Customer opens the application.



Step 2

Customer enters:

- Pickup location
- Destination



Step 3

The system displays available cab types.

Example

- Mini
- Micro
- Sedan
- Prime
- Shared
- Auto

↓

Step 4

Customer selects the preferred cab.

↓

Step 5

The AI system searches for nearby available drivers.

↓

Step 6

The application calculates:

- Estimated Fare
- Estimated Arrival Time (ETA)
- Estimated Travel Time

↓

Step 7

Driver accepts the request.

↓

Step 8

Ride begins.

↓

Step 9

Customer reaches destination.

↓

Step 10

Payment is completed.

↓

Ride Successfully Completed.

Algorithm:

Algorithm OLA_Cab_Booking

Start

Open OLA Application

Input Pickup Location

Input Destination

Display Available Cab Types

Customer Selects Cab Type

Search Nearby Drivers

If Driver Available Then

 Calculate Fare

 Calculate ETA

 Display Driver Details

 If Customer Confirms Then

 Book Ride

 Notify Driver

 Start Ride

 Reach Destination

 Make Payment

 Else

 Cancel Booking

Else

 Search Again

Stop

Pseudocode

Procedure BookCab()

Input Pickup

Input Destination

Display Cab Types

Select Cab

Search Driver

If Driver Exists Then

 Fare ← CalculateFare()

 ETA ← CalculateETA()

 Display Fare

 Display ETA

 If Confirmed Then

 Assign Driver

 Start Ride

 Complete Ride

 Generate Bill

 Else

 Cancel Ride

Else

 Display "No Drivers Available"

End Procedure

Performance Measure

The success of the OLA AI agent is evaluated using the following parameters:

Performance Metric	Description
Waiting Time	Minimum pickup time
Travel Time	Shortest route

Fare	Lowest possible fare
Customer Satisfaction	Comfort and safety
Driver Availability	Fast allocation
Fuel Efficiency	Minimum fuel consumption
Ride Completion	Successful completion of journey

Advantages of AI in OLA Cab Booking

- Fast driver allocation.
- Intelligent route optimization.
- Dynamic fare estimation.
- Traffic prediction.
- Reduced waiting time.
- Better customer experience.
- Efficient use of available drivers.
- Real-time GPS tracking.

Applications

Artificial Intelligence in ride-hailing applications is used for:

- OLA
- Uber
- Rapido
- Lyft
- Public Transport Systems
- Autonomous Vehicle Navigation
- Logistics and Delivery Services

Conclusion

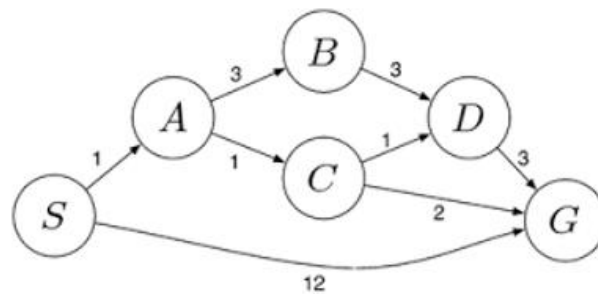
The **OLA Cab Booking System** is a practical example of a **Goal-Based Problem Solving Agent** in Artificial Intelligence. The agent receives the customer's travel request, searches for available drivers, compares available options, estimates fare and travel time, and selects the most appropriate cab to achieve the goal of reaching the destination safely and efficiently. AI techniques such as GPS navigation, route optimization, traffic prediction, and dynamic pricing help improve customer satisfaction while minimizing travel time and operational costs.

Question 5: Uniform Cost Search (UCS) – Finding the Least Cost Path

Problem Statement

A logistics company operates a delivery network where packages are transported between warehouses. Each route has an associated travel cost (fuel cost + travel time). The company wants to determine the **least-cost path** from the **starting warehouse S** to the **destination warehouse G** using the **Uniform Cost Search (UCS)** algorithm.

The delivery network is represented by the following weighted graph:



Edge Costs:

- $S \rightarrow A = 1$
- $S \rightarrow G = 12$
- $A \rightarrow B = 3$
- $A \rightarrow C = 1$
- $B \rightarrow D = 3$
- $C \rightarrow D = 1$
- $C \rightarrow G = 2$
- $D \rightarrow G = 3$

Introduction

Uniform Cost Search (UCS) is an **uninformed search algorithm** that always expands the node with the **lowest cumulative path cost**. Unlike Breadth-First Search (BFS), UCS considers the **cost of each edge** rather than simply the number of steps.

UCS guarantees an **optimal solution** if all edge costs are positive.

Applications include:

- GPS Navigation
- Route Planning
- Logistics Management
- Network Routing
- Robot Navigation
- Delivery Optimization

Problem Formulation

AI problems are represented using the following components.

1. Initial State

The starting warehouse is

S

2. Goal State

The destination warehouse is

G

3. State Space

Each node in the graph represents a warehouse.

Each edge represents a route with an associated travel cost.

4. Operators

Possible actions are:

- Move from one warehouse to another connected warehouse.
- Add the travel cost.
- Expand the least-cost node.

5. Path Cost

The total cost is the sum of all edge costs from the start node to the goal node.

Example:

$S \rightarrow A \rightarrow C \rightarrow G$

$$1 + 1 + 2$$

$$= 4$$

Uniform Cost Search Algorithm

Uniform Cost Search expands nodes in **ascending order of cumulative cost**.

Step 1

Frontier

Node	Cost
S	0

Expand S

New Nodes

Node	Cost
A	1
G	12

Step 2

Choose lowest cost node.

Expand A (**Cost = 1**)

Generate

Node	Cost
B	4
C	2

Frontier becomes

Node	Cost
C	2
B	4
G	12

Step 3

Expand **C (Cost = 2)**

Generate

Node	Cost
D	3
G	4

Frontier

Node	Cost
D	3
B	4
G	4
G	12

Step 4

Expand **D (Cost = 3)**

Generate

Node	Cost
G	6

Frontier

Node	Cost
B	4
G	4
G	6
G	12

Step 5

Expand **B (Cost = 4)**

Generate

Node	Cost
------	------

D	7
---	---

Since **D** already exists with a lower cost (**3**), this path is discarded.

Step 6

The next lowest-cost node is

G (Cost = 4)

Since G is the goal node, the algorithm terminates.

Node Expansion Order

S

↓

A

↓

C

↓

D

↓

B

↓

G

Optimal Path

S

↓

A

↓

C

↓

G

Cost Calculation

Route Cost

$S \rightarrow A$ 1

$A \rightarrow C$ 1

$C \rightarrow G$ 2

Total Cost

$$1 + 1 + 2 = 4$$

Optimal Path: $S \rightarrow A \rightarrow C \rightarrow G$

Minimum Cost: 4

Priority Queue Table

Iteration	Expanded Node	Priority Queue
1	S	A(1), G(12)
2	A	C(2), B(4), G(12)
3	C	D(3), B(4), G(4), G(12)
4	D	B(4), G(4), G(6), G(12)
5	B	G(4), G(6), G(12)
6	G	Goal Found

Algorithm

Algorithm UniformCostSearch

Start

Insert Start Node into Priority Queue

Assign Cost = 0

While Queue is not Empty

 Remove node with Lowest Cost

If Node is Goal

Return Solution

Expand Node

Calculate Cost of Successors

Insert Successors into Queue

End While

Stop

Pseudocode

Procedure UCS(Graph, Start, Goal)

Create priority Queue

Insert(Start,0)

While Queue is not Empty

Current ← Remove Minimum Cost Node

If Current = Goal

Return Path

For each Successor

NewCost = CurrentCost + EdgeCost

If Better Path Exists

Update Queue

End Procedure

Advantages of Uniform Cost Search

- Guarantees the optimal (least-cost) solution.
- Works well for weighted graphs.
- Complete algorithm if costs are positive.
- Widely used in route planning and navigation.
- Considers actual travel costs rather than just the number of steps.

Disadvantages of Uniform Cost Search

- Requires more memory than DFS.
- Can be slower for very large graphs.
- Expands many nodes before reaching the goal.
- Performance depends on the branching factor and edge costs.

Applications of Uniform Cost Search

Uniform Cost Search is used in:

- GPS Navigation Systems
- Google Maps
- Logistics and Supply Chain Management
- Robot Motion Planning
- Airline Route Optimization
- Network Routing Protocols
- Autonomous Vehicles
- Delivery Management Systems

Comparison with Other Search Algorithms

Algorithm	Uses Cost?	Optimal?	Complete?
Breadth First Search	✗ No	Only if equal edge costs	Yes
Depth First Search	✗ No	No	No
Uniform Cost Search	✓ Yes	Yes	Yes
Greedy Best First Search	Uses heuristic only	No	No
A* Search	Cost + Heuristic	Yes	Yes (with admissible heuristic)

Conclusion

Uniform Cost Search (UCS) is an effective search algorithm for solving least-cost path problems in Artificial Intelligence. By expanding the node with the smallest cumulative cost, UCS guarantees the optimal solution when all edge costs are positive. In this logistics network, UCS successfully identifies the optimal route $S \rightarrow A \rightarrow C \rightarrow G$ with a **minimum total cost of 4**. This makes UCS highly suitable for real-world applications such as transportation systems, logistics, navigation, robotics, and network routing, where minimizing travel cost or distance is essential.